

# 14. 上位 MPC 距離メッシュ生成

solar\_ws0129-main

2026-07-29

## Contents

|                                               |          |
|-----------------------------------------------|----------|
| <b>1 対象</b>                                   | <b>3</b> |
| <b>2 このファイルを読む理由</b>                          | <b>3</b> |
| <b>3 呼び出し関係</b>                               | <b>3</b> |
| 3.1 どこから呼ばれるか                                 | 3        |
| 3.2 次に読むと理解が進むファイル                            | 3        |
| 3.3 同一リポジトリ内の主要依存                             | 3        |
| <b>4 ソース冒頭の把握</b>                             | <b>3</b> |
| 4.1 代表コード断片                                   | 3        |
| <b>5 主要構造の読み取り</b>                            | <b>4</b> |
| 5.1 主要クラス・関数                                  | 4        |
| 5.2 ファイルを上から読んだときの定義順                         | 4        |
| 5.3 import 群の詳細                               | 4        |
| <b>6 このファイルを読む前に必要な基礎知識</b>                   | <b>5</b> |
| 6.1 プログラム、プロセス、メモリ、オブジェクトを区別する                | 5        |
| 6.2 名前、代入、型、参照                                | 5        |
| 6.3 関数、引数、戻り値、スコープ                            | 6        |
| 6.4 module、package、import、console_scripts     | 6        |
| 6.5 例外、try/finally、with、資源解放                  | 6        |
| 6.6 class、独自クラス、インスタンス、self、継承                | 6        |
| 6.7 先頭アンダースコア、__init__、名前付けの作法                | 7        |
| 6.8 デコレータと @dataclass                         | 7        |
| 6.9 list、dict、deque、NumPy 配列、pandas DataFrame | 8        |
| 6.10 制御、状態、入力、モデル予測制御 MPC                     | 8        |
| 6.11 天候、route、補間、時刻、単位                        | 8        |
| <b>7 関数・クラスを上から順に解説</b>                       | <b>8</b> |
| 7.1 L14 クラス UpperDistanceHorizon              | 8        |
| 7.2 L20 関数 UpperDistanceHorizon.total_km      | 9        |

|     |                                                            |    |
|-----|------------------------------------------------------------|----|
| 7.3 | L24 関数 <code>_weighted_extra</code> . . . . .              | 10 |
| 7.4 | L33 関数 <code>_adaptive_ds_sequence</code> . . . . .        | 11 |
| 7.5 | L82 関数 <code>build_upper_distance_horizon</code> . . . . . | 13 |
| 7.6 | L136 関数 <code>plan_segment_index</code> . . . . .          | 15 |
| 8   | 処理の流れ                                                      | 16 |
| 9   | このファイルの位置づけ                                                | 16 |

## 1 対象

| 項目          | 内容                                                               |
|-------------|------------------------------------------------------------------|
| ファイル        | mpc_solarcar/upper_horizon.py                                    |
| ソース SHA-256 | 1565f2a9907d31550fd0ee8a54d65bfc840b33c804a95b2c0d24ca9d232baa01 |
| 種別          | Python                                                           |
| 区分          | planner helper                                                   |
| 役割          | 固定または適応距離メッシュを作り、現在地点から先の control point と segment を決める。          |
| 起動文脈        | distance-domain upper planner の最初の一步。                            |

## 2 このファイルを読む理由

固定または適応距離メッシュを作り、現在地点から先の control point と segment を決める。

- build\_upper\_distance\_horizon が中心。
- plan\_segment\_index が現在位置から有効速度区間を引く。

## 3 呼び出し関係

### 3.1 どこから呼ばれるか

- mpc\_node.py
- scripts/solar\_sim.py

### 3.2 次に読むと理解が進むファイル

- mpc\_solarcar/upper\_policy.py

### 3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

## 4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

### 4.1 代表コード断片

```
@dataclass
class UpperDistanceHorizon:
    ds_seq_km: np.ndarray
    seg_s_km: np.ndarray
    ctrl_s_km: np.ndarray

    @property
    def total_km(self) -> float:
```

```

        return float(np.sum(self.ds_seq_km)) if len(self.ds_seq_km) else 0.0

def _weighted_extra(count: int, growth: float) -> np.ndarray:
    if count <= 1:
        return np.ones(max(1, count), dtype=float)
    growth = max(1.0, float(growth))
    if abs(growth - 1.0) <= 1.0e-9:
        return np.ones(count, dtype=float)
    return np.power(growth, np.arange(count, dtype=float))

```

## 5 主要構造の読み取り

主要クラスは UpperDistanceHorizon。主要関数は total\_km, build\_upper\_distance\_horizon, plan\_segment\_index。

### 5.1 主要クラス・関数

| 行   | 種別       | 名前                           | 補足                        |
|-----|----------|------------------------------|---------------------------|
| 14  | class    | UpperDistanceHorizon         |                           |
| 20  | function | total_km                     | args: self                |
| 24  | function | _weighted_extra              | args: count, growth       |
| 33  | function | _adaptive_ds_sequence        | args: target_km           |
| 82  | function | build_upper_distance_horizon |                           |
| 136 | function | plan_segment_index           | args: plan_segments, s_km |

### 5.2 ファイルを上から読んだときの定義順

| 行   | 内容                                     |
|-----|----------------------------------------|
| 10  | MIN_DISTANCE_STEP_KM に 1e-06 の結果を代入する。 |
| 14  | クラス UpperDistanceHorizon を定義する。        |
| 24  | 関数 _weighted_extra を定義する。              |
| 33  | 関数 _adaptive_ds_sequence を定義する。        |
| 82  | 関数 build_upper_distance_horizon を定義する。 |
| 136 | 関数 plan_segment_index を定義する。           |

### 5.3 import 群の詳細

| 行 | import 文                              | このファイルで読む理由                                                                   |
|---|---------------------------------------|-------------------------------------------------------------------------------|
| 1 | from __future_<br>_import annotations | 型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。  |
| 3 | import math                           | Python 標準のスカラー数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での主な使用位置は L48, L112。 |

|   |                                                |                                                                                                                       |
|---|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| 4 | <code>from dataclasses import dataclass</code> | <code>dataclasses</code> から <code>dataclass</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L13。                      |
| 5 | <code>from typing import Iterable, List</code> | <code>typing</code> から <code>Iterable</code> , <code>List</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L136, L137。 |
| 7 | <code>import numpy as np</code>                | 数値配列、 <code>clip</code> 、補間、統計計算を行うため。このファイル内での主な使用位置は L15, L16, L17, L21, L24, L26, L29, L30, ...。                   |

## 6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

### 6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Python インタプリタが読み込む -> OS 上のプロセス -> Python オブジェクトをメモリに生成 -> 関数や callback を実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# node と alias は同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ `self.z` を書き換える場合は実行順と排他を検討する必要がある。

#### 根拠資料

- [Python 公式チュートリアル: Classes](#)

### 6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘`x = f()`’では、まず `f` を呼び、その戻り値が得られてから `x` が更新される。

‘`float(...)`’、‘`int(...)`’、‘`bool(...)`’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘`None`’は値が無いことを示す単一のオブジェクト、‘`math.nan`’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘`is None`’と ‘`math.isfinite`’を使い分ける。

#### 根拠資料

- [Python 公式チュートリアル: Classes](#)

## 6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘\*args’、‘\*\*kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが‘self.z’を更新すればオブジェクトの状態が残るが、単に‘z’を更新しただけならその呼出しのローカル値である。

### 根拠資料

- [Python 公式チュートリアル: Classes](#)

## 6.4 module、package、import、console\_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の‘console\_scripts’は、端末で使う実行可能名と‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼び
```

### 根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

## 6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

## 6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘\_\_init\_\_’ へ渡す。‘self’ は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().\_\_init\_\_(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘create\_publisher’などを正しく使えない。

## 根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

## 6.7 先頭アンダースコア、\_\_init\_\_、名前付けの作法

‘self.step\_solar’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘\_\_init\_\_’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘\_’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘\_base\_csv’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

## 根拠資料

- [Python 公式チュートリアル: Classes](#)

## 6.8 デコレータと @dataclass

‘@名前’は、直後に定義した関数またはクラスを別の関数へ渡し、その結果で定義名を置き換えるデコレータ構文である。‘@Class’という一般構文があるのではなく、‘@dataclass’など実際のデコレータ名を書く。

‘@dataclass’は型注釈付きフィールドから ‘\_\_init\_\_’、‘\_\_repr\_\_’、比較処理などを自動生成する。物理パラメータや解析結果のように、名前付きデータを一まとまりで運ぶ用途に向く。

```
from dataclasses import dataclass

@dataclass
class State:
    soc: float
    temperature_c: float

state = State(soc=0.8, temperature_c=30.0)
```

自動生成は検証を自動で保証しない。単位、許容範囲、相互依存制約は ‘\_\_post\_init\_\_’ や別の検証関数で確認する必要がある。

## 根拠資料

- [Python 公式ライブラリ: dataclasses](#)
- [Python 公式チュートリアル: Classes](#)

## 6.9 list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常 ‘(N,)’、候補集団なら ‘(population, N)’ となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

## 6.10 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を  $x$ 、操作入力を  $u$ 、外乱・予報を  $w$  とすると、離散モデルは ‘ $x[k+1] = f(x[k], u[k], w[k])$ ’ と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から  $N$  ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが receding horizon である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

## 根拠資料

- [SciPy 公式: scipy.optimize.minimize](#)

## 6.11 天候、route、補間、時刻、単位

予報は時刻だけの系列か、時刻と route 距離の 2 次元 grid になり得る。現在時刻  $t$  と距離  $s$  が grid 点の間なら、周囲の値から補間して日射、温度、風を求める。

UTC、現地 timezone、naive datetime を混ぜると数時間のずれが発生する。入力で timezone を確定し、内部比較は UTC、表示は現地時刻という役割分担が安全である。

route 距離の km、速度の km/h と m/s、時間の秒、energy の Wh と J を跨ぐため、変換係数 3.6、3600、1000 の意味を各式で確認する。

## 7 関数・クラスを上から順に解説

### 7.1 L14 クラス UpperDistanceHorizon

- 行範囲: L14–L21



- 所属: module 直下
- 定義: UpperDistanceHorizon(bases=None)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

### この定義を読むための Python 構文

- class 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- @ で始まる行は、定義したクラスを加工する decorator である。

### 上から順の処理

1. ds\_seq\_km にを代入する。
2. seg\_s\_km にを代入する。
3. ctrl\_s\_km にを代入する。
4. 関数 total\_km を定義する。

#### 代表コード断片

```
class UpperDistanceHorizon:
    ds_seq_km: np.ndarray
    seg_s_km: np.ndarray
    ctrl_s_km: np.ndarray

    @property
    def total_km(self) -> float:
        return float(np.sum(self.ds_seq_km)) if len(self.ds_seq_km) else 0.0
```

## 7.2 L20 関数 UpperDistanceHorizon.total\_km

- 行範囲: L20–L21
- 所属: UpperDistanceHorizon
- 定義: total\_km(self) -> float
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: float, len, sum
- 戻り値の要点: float(np.sum(self.ds\_seq\_km)) if len(self.ds\_seq\_km) else 0.0
- 主なローカル代入名: 特になし。

- 読み取る主なインスタンス属性: `self.ds_seq_km`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

### この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `@` で始まる行は、定義した関数を別の関数へ渡して加工する `decorator` である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

### 上から順の処理

1. `float(np.sum(self.ds_seq_km)) if len(self.ds_seq_km) else 0.0` を返す。

#### 代表コード断片

```
def total_km(self) -> float:
    return float(np.sum(self.ds_seq_km)) if len(self.ds_seq_km) else 0.0
```

## 7.3 L24 関数 `_weighted_extra`

- 行範囲: L24–L30
- 所属: `module` 直下
- 定義: `_weighted_extra(count:int,growth:float)->np.ndarray`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `arange`, `float`, `max`, `ones`, `power`
- 戻り値の要点: `np.power(growth, np.arange(count, dtype=float)) / np.ones(max(1, count), dtype=float) / np.ones(count, dtype=float)`
- 主なローカル代入名: `growth`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

### この定義を読むための Python 構文

- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

## 上から順の処理

1. 条件 `count <= 1` を判定し、真なら内部処理を行う。
2. `np.ones(max(1, count), dtype=float)` を返す。
3. `growth` に `max(1.0, float(growth))` の結果を代入する。
4. 条件 `abs(growth - 1.0) <= 1e-09` を判定し、真なら内部処理を行う。
5. `np.ones(count, dtype=float)` を返す。
6. `np.power(growth, np.arange(count, dtype=float))` を返す。

### 代表コード断片

```
def _weighted_extra(count: int, growth: float) -> np.ndarray:
    if count <= 1:
        return np.ones(max(1, count), dtype=float)
    growth = max(1.0, float(growth))
    if abs(growth - 1.0) <= 1.0e-9:
        return np.ones(count, dtype=float)
    return np.power(growth, np.arange(count, dtype=float))
```

## 7.4 L33 関数 `_adaptive_ds_sequence`

- 行範囲: L33–L79
- 所属: module 直下
- 定義: `_adaptive_ds_sequence(target_km:float, *, max_steps:int, min_ds_km:float, max_ds_km:float, growth:float) -> np.ndarray`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_weighted_extra`, `any`, `array`, `ceil`, `copy`, `enumerate`, `flatnonzero`, `float`, `full`, `int`, `max`, `maximum`
- 戻り値の要点: `ds / np.array([target_km], dtype=float) / np.array([target_km], dtype=float) / np.full(step_count, target_km / step_count, dtype=float)`
- 主なローカル代入名: `active`, `active_idx`, `alloc_weights`, `base`, `base_sum`, `cap_value`, `caps`, `ds`, `extra_remaining`, `idx`, `j`, `min_ds_km`, `proposal`, `step_count`, `take`, `target_km`, `used`, `weights`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 5、ループ 2、`try` 0

### この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

## 上から順の処理

1. `target_km` に `max(MIN_DISTANCE_STEP_KM, float(target_km))` の結果を代入する。
2. `min_ds_km` に `max(MIN_DISTANCE_STEP_KM, float(min_ds_km))` の結果を代入する。
3. 条件 `max_steps <= 1` を判定し、真なら内部処理を行う。
4. `np.array([target_km], dtype=float)` を返す。
5. 条件 `target_km <= min_ds_km` を判定し、真なら内部処理を行う。
6. `np.array([target_km], dtype=float)` を返す。
7. `step_count` に `int(min(max_steps, max(1, math.ceil(target_km / min_ds_km))))` の結果を代入する。
8. `base` に `np.full(step_count, min_ds_km, dtype=float)` の結果を代入する。
9. `base_sum` に `float(base.sum())` の結果を代入する。
10. 条件 `target_km <= base_sum + 1e-09` を判定し、真なら内部処理を行う。
11. `np.full(step_count, target_km / step_count, dtype=float)` を返す。
12. `weights` に `_weighted_extra(step_count, growth)` の結果を代入する。
13. `weights` に `weights / max(float(weights.sum()), 1e-09)` の結果を代入する。
14. `ds` に `base.copy()` の結果を代入する。
15. `extra_remaining` に `float(target_km - base_sum)` の結果を代入する。
16. `cap_value` に `max(float(max_ds_km), float(min_ds_km))` の結果を代入する。
17. `caps` に `np.maximum(0.0, cap_value - ds)` の結果を代入する。
18. `active` に `caps > 1e-09` の結果を代入する。
19. 条件 `extra_remaining > 1e-09 and np.any(active)` が成り立つ間くり返す。
20. `active_idx` に `np.flatnonzero(active)` の結果を代入する。
21. `alloc_weights` に `weights[active_idx]` の結果を代入する。
22. `alloc_weights` に `alloc_weights / max(float(alloc_weights.sum()), 1e-09)` の結果を代入する。
23. `proposal` に `extra_remaining * alloc_weights` の結果を代入する。
24. `used` に `0.0` の結果を代入する。
25. `enumerate(active_idx)` を順に走査し、各要素を `(j, idx)` に入れて処理する。
26. `take` に `min(float(proposal[j]), float(caps[idx]))` の結果を代入する。
27. `ds[idx]` を Add で更新する。
28. `caps[idx]` を Sub で更新する。
29. `used` を Add で更新する。
30. `extra_remaining` を Sub で更新する。
31. `active` に `caps > 1e-09` の結果を代入する。
32. 条件 `used <= 1e-09` を判定し、真なら内部処理を行う。
33. Break 文を実行する。

34. 条件 `extra_remaining > 1e-09` を判定し、真なら内部処理を行う。
35. `ds[-1]` を `Add` で更新する。
36. `ds` を返す。

#### 代表コード断片

```
def _adaptive_ds_sequence(
    target_km: float,
    *,
    max_steps: int,
    min_ds_km: float,
    max_ds_km: float,
    growth: float,
) -> np.ndarray:
    target_km = max(MIN_DISTANCE_STEP_KM, float(target_km))
    min_ds_km = max(MIN_DISTANCE_STEP_KM, float(min_ds_km))
    if max_steps <= 1:
        return np.array([target_km], dtype=float)
    if target_km <= min_ds_km:
        return np.array([target_km], dtype=float)

    step_count = int(min(max_steps, max(1, math.ceil(target_km / min_ds_km))))
    base = np.full(step_count, min_ds_km, dtype=float)
    base_sum = float(base.sum())
    if target_km <= base_sum + 1.0e-9:
        return np.full(step_count, target_km / step_count, dtype=float)

    weights = _weighted_extra(step_count, growth)
    weights = weights / max(float(weights.sum()), 1.0e-9)
    ds = base.copy()
    extra_remaining = float(target_km - base_sum)
    cap_value = max(float(max_ds_km), float(min_ds_km))
    caps = np.maximum(0.0, cap_value - ds)
    active = caps > 1.0e-9
    while extra_remaining > 1.0e-9 and np.any(active):
        active_idx = np.flatnonzero(active)
        alloc_weights = weights[active_idx]
        alloc_weights = alloc_weights / max(float(alloc_weights.sum()), 1.0e-9)
        proposal = extra_remaining * alloc_weights
        used = 0.0
        for j, idx in enumerate(active_idx):
            ...
```

## 7.5 L82 関数 `build_upper_distance_horizon`

- 行範囲: L82–L133
- 所属: module 直下
- 定 義: `build_upper_distance_horizon(*, mode:str, s0_km:float, race_km:float, ds_km:float, horizon_km:float, max_steps:int, ctrl_km:float, adaptive_min_ds_km:float, adaptive_max_ds_km:float, adaptive_growth:float) -> UpperDistanceHorizon`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `UpperDistanceHorizon`, `_adaptive_ds_sequence`, `append`, `arange`, `array`, `ceil`, `concatenate`, `cumsum`, `float`, `full`, `int`, `len`
- 戻り値の要点: `UpperDistanceHorizon(ds_seq_km=ds_seq, seg_s_km=seg_s, ctrl_s_km=ctrl_s)`

- 主なローカル代入名: `control_end_km`, `covered_before_last`, `ctrl_s`, `ctrl_step_km`, `ds_km`, `ds_seq`, `horizon_km`, `max_steps`, `mode`, `remaining_km`, `seg_s`, `step_count`, `target_km`, `total_km`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 4、ループ 0、`try` 0

## この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

## 上から順の処理

1. `ds_km` に `max(MIN_DISTANCE_STEP_KM, float(ds_km))` の結果を代入する。
2. `horizon_km` に `max(ds_km, float(horizon_km))` の結果を代入する。
3. `remaining_km` に `max(0.0, float(race_km) - float(s0_km))` の結果を代入する。
4. `max_steps` に `max(1, int(max_steps))` の結果を代入する。
5. `mode` に `str(mode or 'fixed').strip().lower()` の結果を代入する。
6. 条件 `mode in {'adaptive_full_race', 'remaining_race', 'full_race'}` を判定し、真なら内部処理を行う。
7. `target_km` に `max(ds_km, remaining_km if remaining_km > 0.0 else horizon_km)` の結果を代入する。
8. `ds_seq` に `_adaptive_ds_sequence(target_km, max_steps=max_steps, min_ds_km=max(ds_km, float(adaptive_min_ds_km)), max_ds_km=max(float(adaptive_max_ds_km), max(ds_km, float(adaptive_min_ds_km))), growth=float(adaptive_growth))` の結果を代入する。
9. 上の条件が偽の場合:
10. `target_km` に `max(ds_km, min(horizon_km, remaining_km if remaining_km > 0.0 else horizon_km))` の結果を代入する。
11. `step_count` に `int(max(1, math.ceil(target_km / ds_km)))` の結果を代入する。
12. `step_count` に `min(step_count, max_steps)` の結果を代入する。
13. `ds_seq` に `np.full(step_count, ds_km, dtype=float)` の結果を代入する。
14. `covered_before_last` に `float(ds_km * max(0, step_count - 1))` の結果を代入する。
15. `ds_seq[-1]` に `max(MIN_DISTANCE_STEP_KM, target_km - covered_before_last)` の結果を代入する。
16. `seg_s` に `np.concatenate([[0.0], np.cumsum(ds_seq[:-1], dtype=float)])` の結果を代入する。
17. `total_km` に `float(np.sum(ds_seq))` の結果を代入する。
18. `control_end_km` に `float(seg_s[-1]) if len(seg_s) else 0.0` の結果を代入する。
19. `ctrl_step_km` に `float(ctrl_km) if ctrl_km and ctrl_km > 0.0 else float(ds_seq[0])` の結果を代入する。
20. `ctrl_step_km` に `max(MIN_DISTANCE_STEP_KM, min(ctrl_step_km, max(control_end_km, MIN_DISTANCE_STEP_KM)))` の結果を代入する。
21. `ctrl_s` に `np.arange(0.0, control_end_km + 1e-09, ctrl_step_km, dtype=float)` の結果を代入する。

22. 条件 `len(ctrl_s) == 0` を判定し、真なら内部処理を行う。
23. `ctrl_s` に `np.array([0.0], dtype=float)` の結果を代入する。
24. 条件 `len(ctrl_s) > len(ds_seq)` を判定し、真なら内部処理を行う。
25. `ctrl_s` に `np.array(seg_s, dtype=float)` の結果を代入する。
26. 条件 `ctrl_s[-1] < control_end_km - 1e-09` を判定し、真なら内部処理を行う。
27. `ctrl_s` に `np.append(ctrl_s, control_end_km)` の結果を代入する。
28. `UpperDistanceHorizon(ds_seq_km=ds_seq, seg_s_km=seg_s, ctrl_s_km=ctrl_s)` を返す。

#### 代表コード断片

```
def build_upper_distance_horizon(
    *,
    mode: str,
    s0_km: float,
    race_km: float,
    ds_km: float,
    horizon_km: float,
    max_steps: int,
    ctrl_km: float,
    adaptive_min_ds_km: float,
    adaptive_max_ds_km: float,
    adaptive_growth: float,
) -> UpperDistanceHorizon:
    ds_km = max(MIN_DISTANCE_STEP_KM, float(ds_km))
    horizon_km = max(ds_km, float(horizon_km))
    remaining_km = max(0.0, float(race_km) - float(s0_km))
    max_steps = max(1, int(max_steps))
    mode = str(mode or "fixed").strip().lower()

    if mode in {"adaptive_full_race", "remaining_race", "full_race"}:
        target_km = max(ds_km, remaining_km if remaining_km > 0.0 else horizon_km)
        ds_seq = _adaptive_ds_sequence(
            target_km,
            max_steps=max_steps,
            min_ds_km=max(ds_km, float(adaptive_min_ds_km)),
            max_ds_km=max(float(adaptive_max_ds_km), max(ds_km, float(adaptive_min_ds_km))),
            growth=float(adaptive_growth),
        )
    else:
        target_km = max(ds_km, min(horizon_km, remaining_km if remaining_km > 0.0 else
horizon_km))
        step_count = int(max(1, math.ceil(target_km / ds_km)))
        step_count = min(step_count, max_steps)
        ds_seq = np.full(step_count, ds_km, dtype=float)
        covered_before_last = float(ds_km * max(0, step_count - 1))
        ds_seq[-1] = max(MIN_DISTANCE_STEP_KM, target_km - covered_before_last)
    ...
```

## 7.6 L136 関数 `plan_segment_index`

- 行範囲: L136–L145
- 所属: module 直下
- 定義: `plan_segment_index(plan_segments:Iterable[dict],s_km:float)->int`
- docstring: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: `enumerate`, `float`, `get`, `len`, `list`
- 戻り値の要点: `len(segments) - 1 / -1 / idx`
- 主なローカル代入名: `idx`, `s_end_km`, `s_km`, `seg`, `segments`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、`try` 0

## この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

## 上から順の処理

1. `segments` に `list(plan_segments)` を代入する。
2. 条件 `not segments` を判定し、真なら内部処理を行う。
3. `-1` を返す。
4. `s_km` に `float(s_km)` の結果を代入する。
5. `enumerate(segments)` を順に走査し、各要素を `(idx, seg)` に入れて処理する。
6. `s_end_km` に `float(seg.get('s_end_km', seg.get('s_start_km', 0.0)))` の結果を代入する。
7. 条件 `s_km < s_end_km - 1e-09` を判定し、真なら内部処理を行う。
8. `idx` を返す。
9. `len(segments) - 1` を返す。

## 代表コード断片

```
def plan_segment_index(plan_segments: Iterable[dict], s_km: float) -> int:
    segments: List[dict] = list(plan_segments)
    if not segments:
        return -1
    s_km = float(s_km)
    for idx, seg in enumerate(segments):
        s_end_km = float(seg.get("s_end_km", seg.get("s_start_km", 0.0)))
        if s_km < s_end_km - 1.0e-9:
            return idx
    return len(segments) - 1
```

## 8 処理の流れ

1. ソース中の主要関数を通じて処理を進める。

## 9 このファイルの位置づけ

この資料群では、`mpc_solarcar/upper_horizon.py` を **planner helper** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。